Era Komputasi Memori: Pengantar Apache Spark & RDD

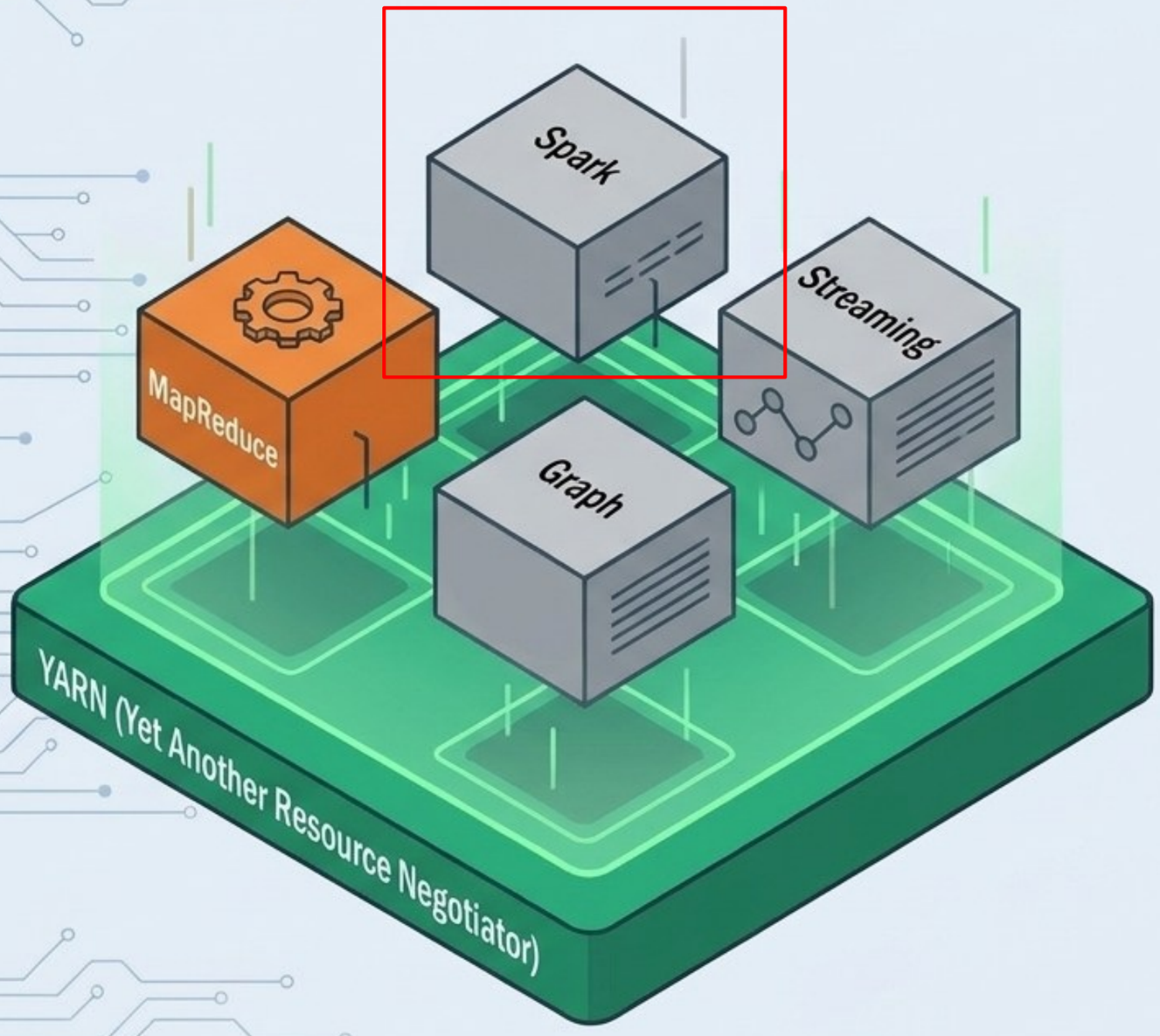
Transisi dari Disk I/O ke Pemrosesan Terdistribusi Berbasis DAG

MATA KULIAH BIG DATA — PROGRAM STUDI S1 TEKNIK INFORMATIKA



Gemini Notebook

YARN: Sistem Operasi Big Data Masa Kini



Sistem manajemen sumber daya **multi-tenant** yang memisahkan Manajemen Sumber Daya dari Penjadwalan Aplikasi.

Evolusi Platform: Hadoop tidak lagi eksklusif untuk batch processing MapReduce. YARN membuka jalan bagi berbagai arsitektur pemrosesan modern untuk berbagi klaster yang sama secara harmonis dan efisien.

Gemini Notebook

Akar Masalah Era Pre-Spark: Keterbatasan Disk I/O



Hadoop MapReduce merevolusi komputasi kluster, namun memiliki kelemahan arsitektural fatal pada pemrosesan iteratif.

Setiap kali satu fase Map atau Reduce selesai, sistem secara otomatis menyimpan hasil sementara (intermediate data) ke disk fisik.

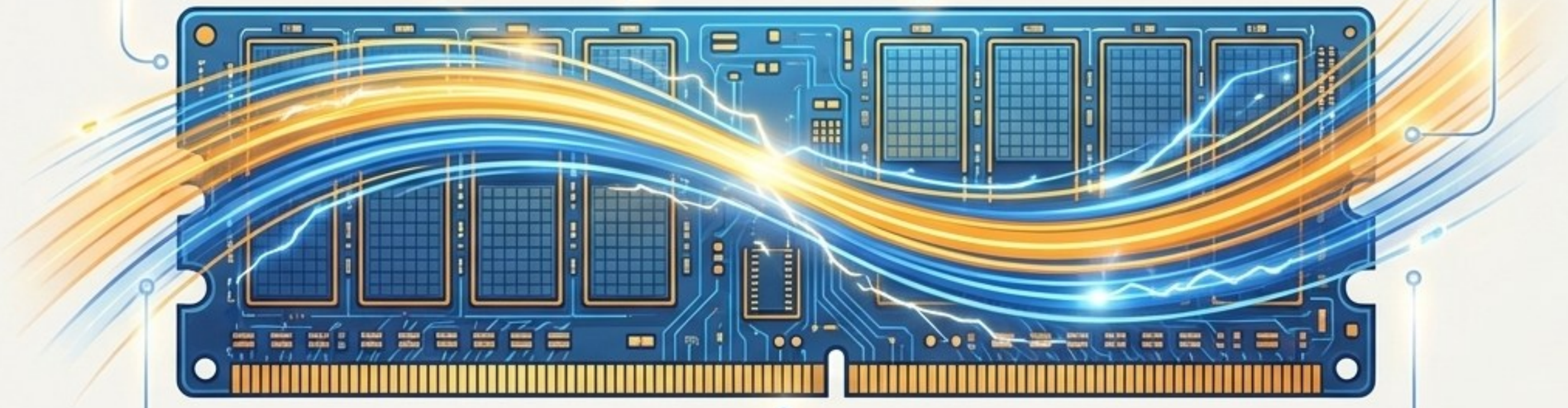
Dampak: Siklus baca/tulis disk yang konstan menciptakan latensi tinggi (I/O bottleneck), menghancurkan performa algoritma Machine Learning.

Gemini Notebook

Paradigma Baru: Pemrosesan In-Memory

Apache Spark lahir di UC Berkeley (2009) dengan filosofi baru: **Pertahankan data di dalam memori (RAM)** selama mungkin.

Menghilangkan **latensi disk I/O**, memberikan **lonjakan kecepatan** komputasi hingga 10x - 100x lipat dibandingkan MapReduce.



Didesain sebagai Unified Computing Engine—satu mesin utama yang menyatukan analitik batch, interaktif, streaming, dan graf.

Gemini Notebook

Matriks Komparasi: Friksi vs. Aliran

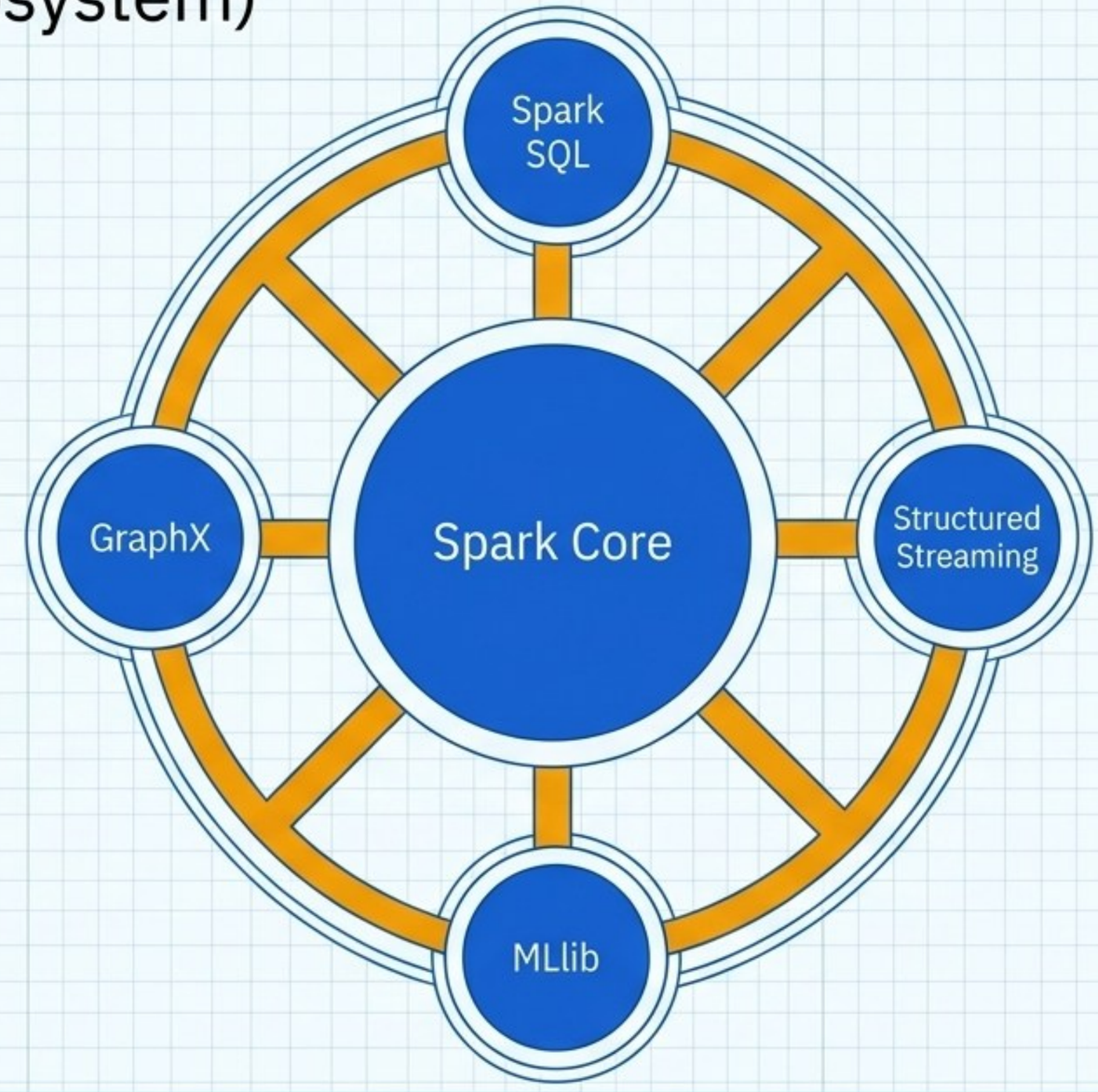
Dimensi Teknis	Hadoop MapReduce (Friksi)	Apache Spark (Aliran)
Penyimpanan Data	Berbasis Disk Fisik (HDFS)	In-Memory Computing (RAM)
Model Eksekusi	Terputus-putus (Batch 2-Tahap)	Kontinu (DAG Pipelining)
Toleransi Kegagalan	Replikasi data yang berat	Lineage Graph (Perhitungan ulang)
Kompleksitas Kode	Sangat tinggi (Verbose Java)	API tingkat tinggi (Composable)

Gemini Notebook

Ekosistem Terpadu (Unified Ecosystem)

Satu Mesin, Berbagai Beban Kerja: Tidak perlu mempelajari sistem terpisah untuk kebutuhan analitik yang berbeda.

- **Spark SQL**: Pemrosesan data terstruktur dengan query SQL yang dioptimalkan.
- **Structured Streaming**: Analitik data real-time dengan semantik yang sama seperti data batch.
- **MLlib**: Perpustakaan Machine Learning terdistribusi berskala besar.
- **GraphX**: Pemrosesan dan analitik graf (jaringan).



Gemini Notebook

Evolusi Pengalaman Developer




```
df = spark.read.json('data.json')
filtered = df.filter(df.age > 21)
filtered.show()
```

Ringkas & Deklaratif

API yang Komposabel: Memungkinkan penggabungan fungsi secara deklaratif dan ringkas.

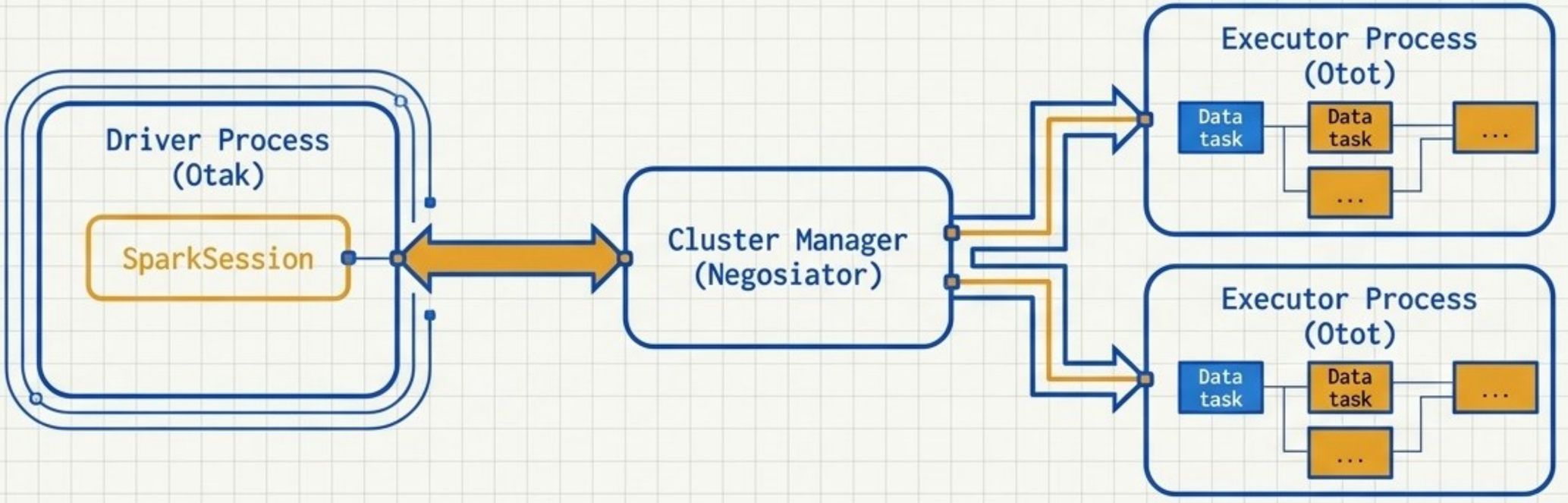
Dukungan Multi-Bahasa: Terintegrasi secara native dengan Scala, Java, Python (PySpark), R, dan SQL.

Interaktivitas: Menyediakan REPL (Read-Eval-Print Loop) untuk eksplorasi data real-time.

Gemini Notebook

Anatomi Kluster Spark: Tiga Serangkai

Aplikasi Spark berjalan sebagai sekumpulan proses independen pada kluster, dikoordinasikan oleh objek SparkSession.



Driver Process:

Otak yang memelihara state aplikasi, merencanakan DAG, dan mendistribusikan pekerjaan.

Cluster Manager:

Negosiator eksternal yang mengalokasikan sumber daya perangkat keras fisik.

Executor Process:

Otot yang mengeksekusi tugas secara paralel dan menyimpan data di dalam memori.

Gemini Notebook

The Driver Process (Otak Sistem)



Proses utama yang berjalan di atas node tempat Anda memanggil fungsi main().

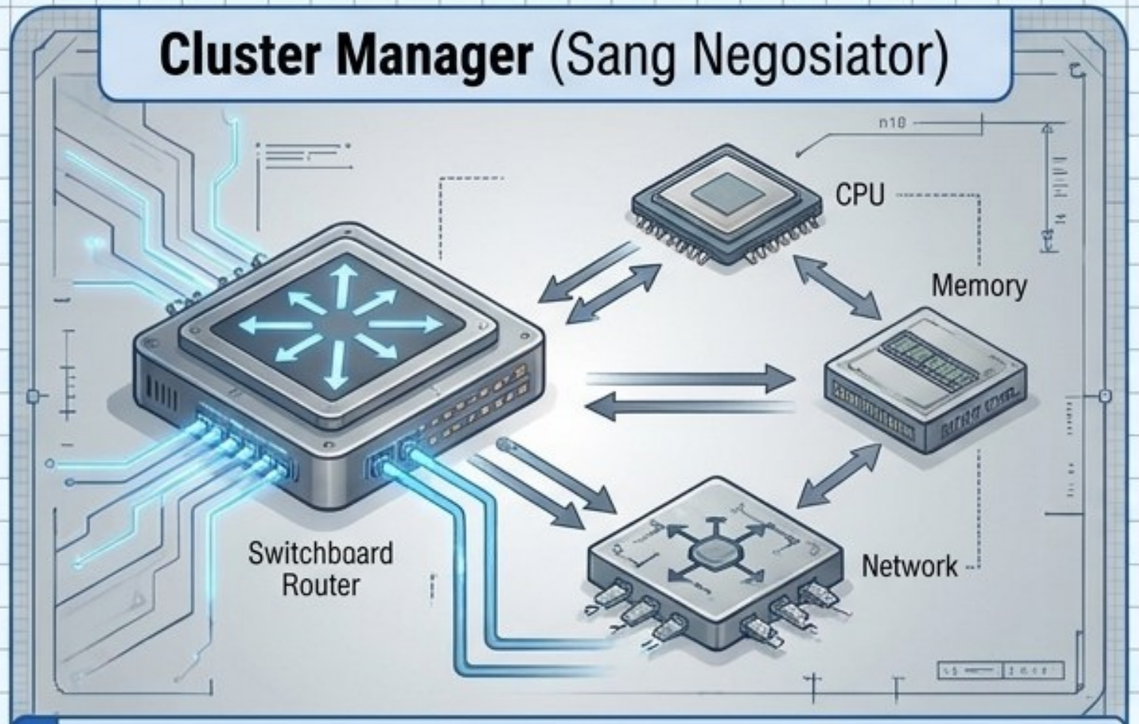
Tugas Utama Driver:

- **State Management**: Memelihara seluruh informasi dan status aplikasi melalui SparkSession atau SparkContext.
- **Translation & Planning:** Menerjemahkan kode program pengguna menjadi grafik tugas (Directed Acyclic Graph / DAG) yang logis.
- **Scheduling:** Menjadwalkan dan mendistribusikan task (tugas) ke Executors yang tersedia di dalam kluster.

Gemini Notebook

Cluster Manager & Executors

Cluster Manager (Sang Negosiator)



- **Berkomunikasi** dengan infrastruktur fisik untuk meminta CPU dan memori.
- **Bersifat** *pluggable*: Mendukung berbagai manajer kluster seperti YARN, Mesos, Kubernetes, atau Standalone.

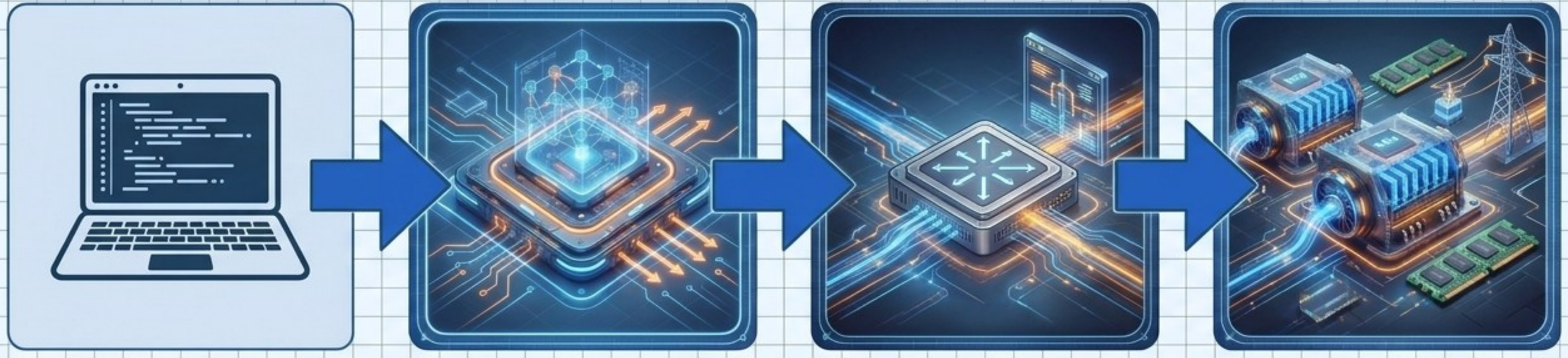
Executors (Otot Pekerja)



- **Mengeksekusi** kode (*task*) yang dikirimkan secara langsung oleh Driver.
- **Menyimpan data** *cache* secara mandiri di dalam memori (RAM) mereka sendiri.
- **Terus-menerus melaporkan** status komputasi dan *heartbeat* kembali ke Driver.

Gemini Notebook

Blueprint: Dari Kode ke Kluster



1. Inisiasi

Pengguna menjalankan spark-submit. Kode aplikasi dievaluasi.

2. Perencanaan

Driver membangun DAG dari operasi logika dan memecahnya menjadi Stages dan Tasks.

3. Negosiasi

Driver meminta resources dari Cluster Manager untuk meluncurkan Executors.

4. Eksekusi

Tasks dikirim ke Executors. Hasil akhir dikembalikan ke Driver atau storage.

Gemini Notebook

Konsep Inti: Resilient Distributed Dataset (RDD)



Apa itu RDD?

- Struktur data logis tingkat rendah (*low-level*) yang menjadi fondasi dasar dari seluruh operasi komputasi Apache Spark.
- Secara sederhana: RDD adalah sekumpulan objek data yang didistribusikan melintasi banyak mesin di dalam kluster, yang dapat diproses secara masif dan paralel.
- **Catatan**: Semua API modern Spark (DataFrame, Dataset) pada akhirnya dikompilasi menjadi RDD oleh sistem di bawah layar.

Gemini Notebook

Tiga Pilar Utama RDD



1. Resilient (Tangguh)

Mampu memulihkan data secara otomatis dari kegagalan node tanpa perlu mereplikasi data ke disk, berkat mekanisme Lineage Graph.

2. Distributed (Terdistribusi)

Data tidak disimpan di satu tempat, melainkan dipartisi dan disebar melintasi puluhan atau ribuan worker nodes di seluruh kluster.

3. Dataset yang Immutable

Sekali RDD dibuat, wujudnya tidak dapat diubah (read-only). Ia hanya bisa ditransformasikan menjadi RDD baru yang berbeda.

Gemini Notebook

Toleransi Kegagalan via Lineage Graph



Pendekatan Lama
Menyalin data 3x ke disk fisik secara redundan untuk keamanan. (Sangat berat dan lambat).

Pendekatan Spark (Lineage Graph)
Mengingat seluruh riwayat instruksi (transformasi). Jika sebuah partisi hilang, Spark cukup melihat Lineage Graph dan menghitung ulang partisi yang hilang dari data induknya. (Ringan dan sangat cepat).

Gemini Notebook

Anatomi Operasi RDD: Dua Sisi Koin



Transformations (Membangun Rencana)

- Menerima RDD dan mengembalikan RDD baru.
- Sifat: Lazy (Malas). Spark tidak memproses data saat itu juga, sistem hanya mencatat instruksinya dalam bentuk DAG.
- Contoh: map(), filter(), flatMap()



Actions (Mengeksekusi Pekerjaan)

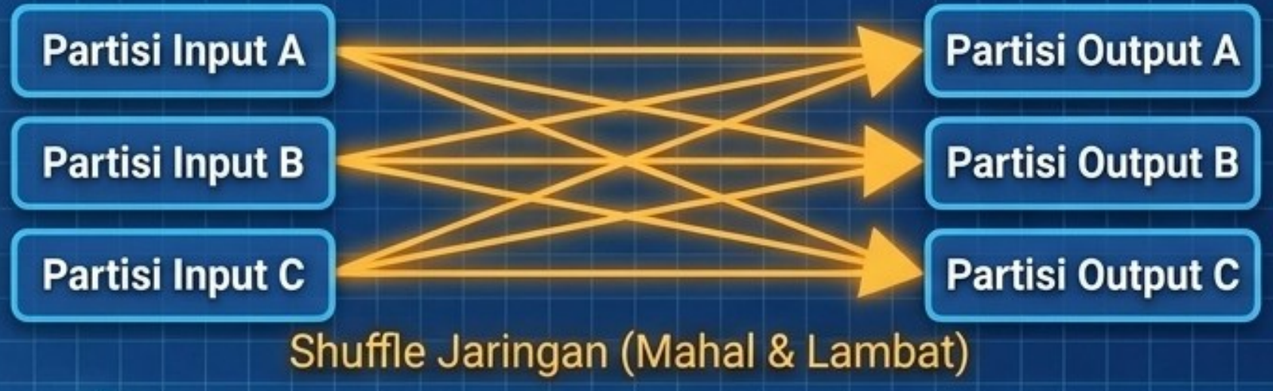
- Mengembalikan nilai akhir ke Driver atau menulis ke penyimpanan eksternal.
- Sifat: Eager (Agresif). Memaksa mesin Spark untuk mulai menjalankan seluruh Transformations yang telah dicatat sebelumnya.
- Contoh: count(), collect(), save()

Gemini Notebook

Transformasi: Narrow vs. Wide Dependencies



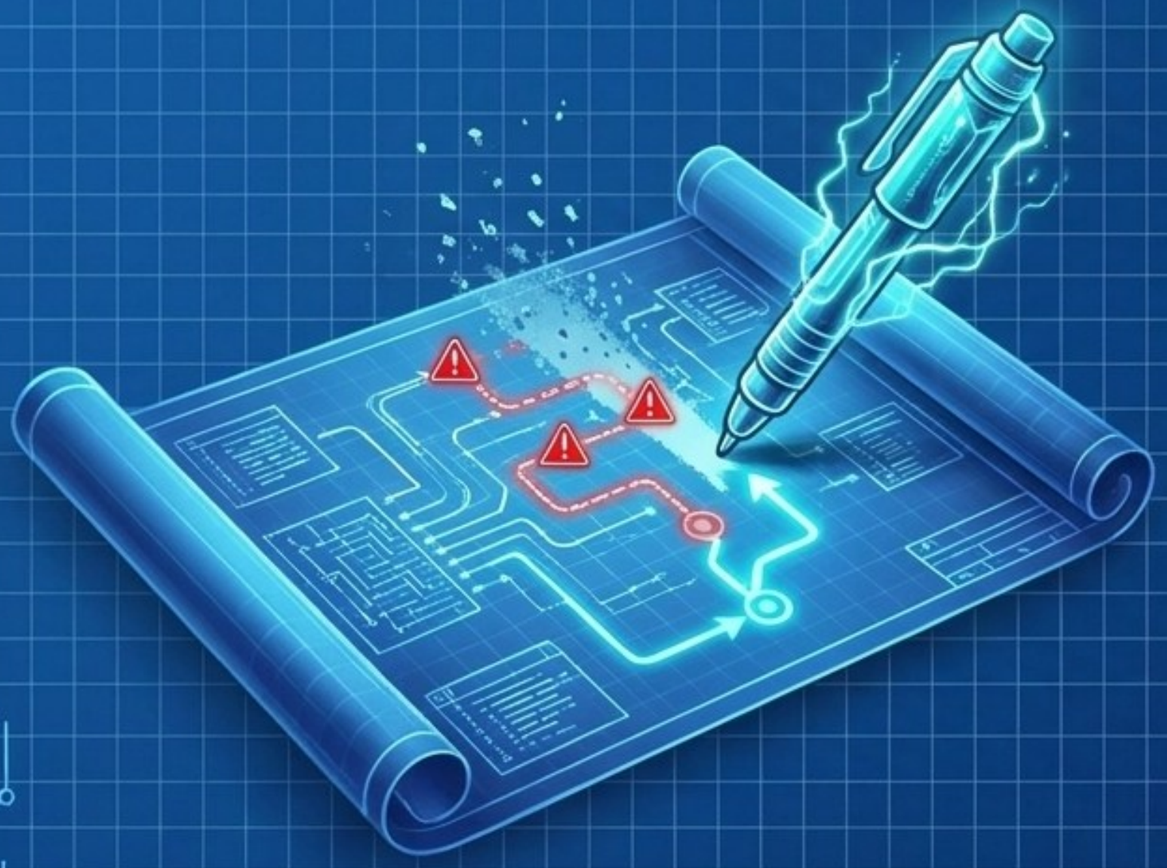
NARROW DEPENDENCIES
Setiap partisi input hanya berkontribusi pada satu partisi output. Tidak ada perpindahan data antar node.
(Contoh: filter, map).



WIDE DEPENDENCIES
Partisi input menyumbangkan data ke banyak partisi output. Membutuhkan sinkronisasi jaringan massal yang disebut Shuffle.
(Contoh: groupByKey, reduceByKey).

Gemini Notebook

Konsep Lazy Evaluation: Kekuatan Menunda Pekerjaan



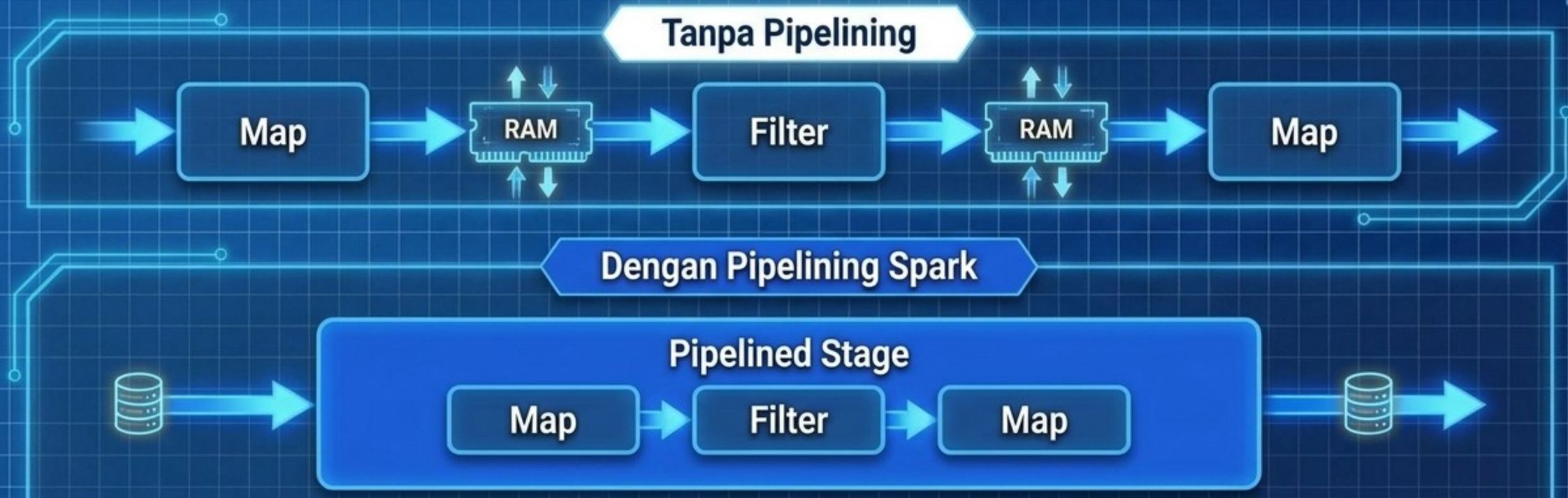
Optimisasi Sebelum Eksekusi Fisik

Mengapa Spark Menunggu Panggilan Action?

1. **Visibilitas End-to-End:** Dengan menunda eksekusi, sistem mendapat pandangan utuh atas seluruh alur kerja.
2. **Optimalisasi Otomatis:** DAG Scheduler (dan Catalyst Optimizer) dapat menyusun ulang instruksi secara logis sebelum dijalankan.
3. **Penghapusan Redundansi:** Sistem dapat menggabungkan filter dan menghapus langkah yang sia-sia sebelum satu byte data pun dipindahkan ke memori.

Gemini Notebook

Optimalisasi Rencana DAG & Pipelining



Pipelining: Operasi Narrow Dependencies yang berurutan digabungkan secara otomatis menjadi satu Stage tunggal.

Daripada menulis hasil sementara ke memori setelah setiap operasi, Spark mengambil satu rekaman data dan melewatkannya ke serangkaian operasi tersebut sekaligus di dalam L1/L2 Cache CPU.

Efisiensi Luar Biasa: Menghindari alokasi memori berlebih dan memangkas siklus komputasi yang tidak perlu.

Gemini Notebook

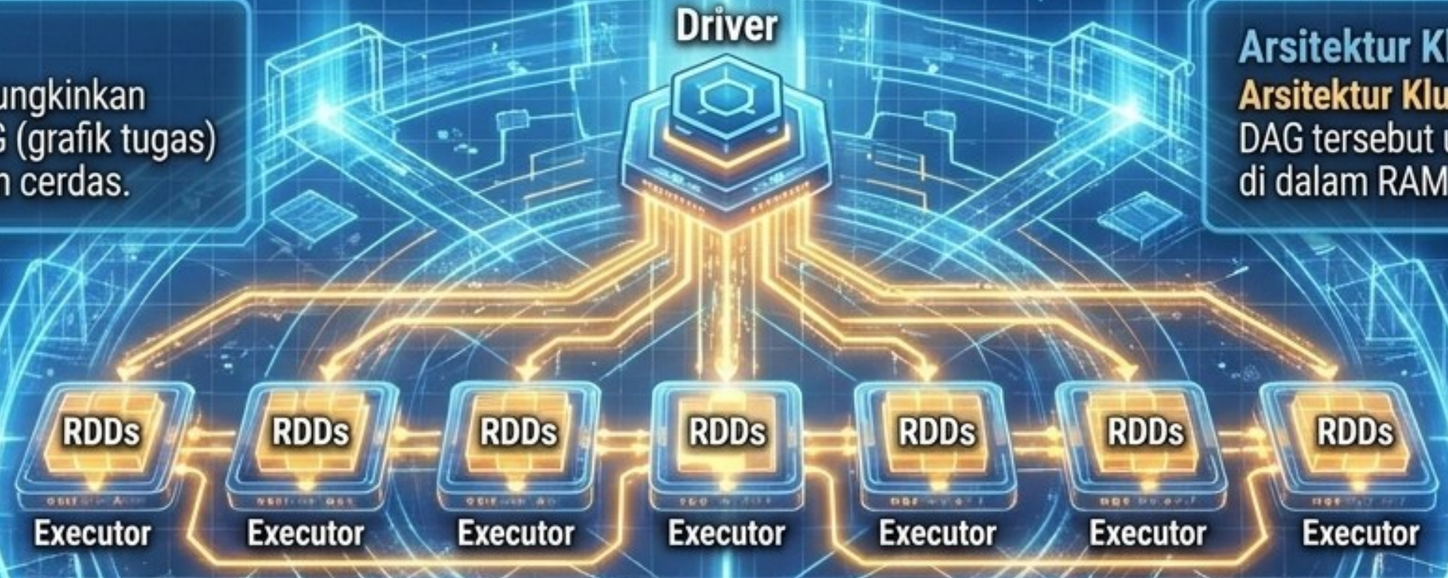
Sintesis: Harmoni Komputasi Kecepatan Cahaya

Lazy Evaluation

Lazy Evaluation memungkinkan Driver merancang DAG (grafik tugas) yang paling efisien dan cerdas.

Arsitektur Kluster

Arsitektur Kluster mendistribusikan DAG tersebut untuk dieksekusi langsung di dalam RAM **Executor** (In-Memory).



Sifat RDD Lineage

Sifat RDD Lineage menjamin seluruh komputasi aman dari kegagalan hardware tanpa perlu replikasi data yang berat.

Hasil Akhir: Engine komputasi terpadu yang membebaskan analitik Big Data dari batasan fisik I/O disk.

Gemini Notebook